$_{a}udit116table.1$

An Extension of Buddy Systems to Arbitrary Radix for Deterministic Memory Allocation

Andrew Jonathan Fine
Independent Student in Computer Architecture
i.am.extremely.useful@gmail.com

April 2026

1 Introduction to Iscandar

Unlike allocation models using variable-time search algorithms [1], this architecture employs a hierarchical occupancy map to resolve coordinates deterministically. Extending the traditional binary buddy system to an arbitrary radix replaces it with a geometric steering mechanism.

2 Architectural Axioms

These axioms establish the geometric and mathematical constraints of the lattice. They transform each memory request into a coordinate resolution task, avoiding search-and-sort operations to maintain a fixed sequence of instructions.

2.1 Axiom I: The State Map (L) and Structural Arity ($\mathcal{A}$)

For modeling and testing purposes this paper demonstrates an k-level lattice L of fixed depth ($n = 3$) **defining** a total memory space. The architecture **maintains** a constant Arity ($\mathcal{A}$) of 32. A hierarchical progression of tiers **organizes** the lattice: the atomic Sector (S_0), the Row of Sectors (S_1), the Plane of Rows (S_2), and the Cube of Planes (S_3). Lattice coordinates (p, r, s) **determine** the address of any specific leaf node, where $p, r, s \in \{0, \ldots, \mathcal{A} - 1\}$. The geometry **maintains** invariance regardless of the physical byte-width the sector **occupies**.[†]

2.2 Axiom II: Hierarchical Occupancy Counters (Γ)

Discrete occupancy counts (Γ) at every tier **enable** constant-time steering. Following bit-block representation principles [2], these counters **generate** a real-time density map of the lattice:

- **Level 1 (Row):** $\Gamma_1 = \sum_{i=0}^{\mathcal{A}-1} \Gamma_{0,i}$
- **Level 2 (Plane):** $\Gamma_2 = \sum_{j=0}^{\mathcal{A}-1} \Gamma_{1,j}$
- **Level 3 (Cube):** $\Gamma_3 = \sum_{k=0}^{\mathcal{A}-1} \Gamma_{2,k}$

This hierarchical summation lets a Global Registrar at the Cube level **execute** an admissibility check in a single instruction cycle, **imposing** a fast-fail condition.

[†]The Large allocator functions as a cross-tier searcher for sequence resolution, managing the fractal logic across the Plane manifold to satistfy multi-Row requests.

2.3 Axiom III: Enforcing an Occupancy Limit (Φ)

The **Iscandar** system **enforces** a mandatory occupancy boundary, the Occupancy Limit (Φ). Traditional allocators search for gaps as memory nears capacity; in contrast, this system **triggers** a deterministic refusal once Γ_3 reaches the Φ threshold based on the present occupancy request.

2.4 Axiom IV: Rotational Steering ($\vec{R}$)

A rotational steering vector **balances** the lattice structure, preventing localized occupancy "hot spots". To prevent high-density clusters that necessitate bit-shuffling or metadata updates, the system utilizes $\vec{R}$ to distribute entropy across the manifold [3].

2.5 Axiom V: The Contiguity Guarantee

Any resolved coordinate sequence **represents** a continuous physical block in the **Iscandar** lattice. An Oracle in the form of look-up tables **validates** that the combined leading and trailing fragments across a sector-join **equal** the requested magnitude M without gaps.

3 Inductive Proof of Deterministic Resolution

Theorem: A fixed-depth $\mathcal{L}$ of Arity $\mathcal{A} = 32$ **establishes** deterministic resolution for operator Ω.

3.1 The Complexity Proof

The $\mathcal{L}$ **maps** its branching factor to the processor word-width to **optimize** the classical buddy system search [3]. Let $\mathcal{L}_n$ **designate** a sub-lattice of level n, where resolution time $T(\mathcal{L}_n)$ **represents** the total instruction count (ι) required to **resolve** a memory coordinate. Resolution time $T(\mathcal{L}_n)$ **represents** the instruction count (ι) for **resolving** a memory coordinate in a level-n sub-lattice [4].

1. **Base Case ($n = 1$):** At the Row level (L_1), a pre-calculated **Instruction Group mapping** resolves a free sector via a static Look-Up Table. This operation maintains atomicity by collapsing the resolution into a single-cycle, table-driven state transition.

$$T(\mathcal{L}_1) = I_{base} \tag{1}$$

2. **Inductive Step:** Given the invariant that resolution time for a level-k structure is $T(\mathcal{L}_k) = kI_{base}$, resolving a request at level $k + 1$ (e.g., the Plane level) **requires**:

   - The occupancy sum Γ_{k+1} is **evaluated** via bitmask comparison to **select** a viable Level-k child. When $\mathcal{A}$ **aligns** with a power of 2, this lookup and steering operation **consumes** a single table lookup [4].
   - The resolution operation **descends** to the level-k $\mathcal{L}$ identified by the steering vector from the Oracle.
   - The resulting recurrence relation is:

$$T(\mathcal{L}_{k+1}) = I_{base} + T(\mathcal{L}_k) = I_{base} + kI_{base} = (k + 1)I_{base} \tag{2}$$

   - **Termination Constraint:** The recursion **terminates** at $\mathcal{L}_1$, where the hardware bit-scan **yields** the final effective address offset.

3.2 Conclusion of Determinism

By induction, the resolution time for any n-level $\mathcal{L}$ structure is strictly nI_{base}. While standard stochastic allocators exhibit execution times as a function of free-block count (N)—typically yielding $O(\log N)$ or worse in fragmented states [5]—the **Iscandar** architecture **collapses** this complexity. n as a constant **enforces** deterministic latency of resolution regardless of heap saturation.

In our model, n is an architectural constant (e.g., $n = 3$); so total execution time is:

$$T_{total} = 3I_{base} \tag{3}$$

This result confirms that for a Cube of $\mathcal{A}^k$ Sectors, the time required to resolve an address is invariant. This performance is maintained provided the system stays in the operational domain of the Occupancy Limit (Φ). By trading spatial capacity for temporal certainty, the architecture **eliminates** the "temporal jitter" that characterizes non-deterministic real-time systems. [5]

3.3 Temporal Consistency and Boundary Management

To **preclude** latency variability during boundary-straddling, **Iscandar utilizes** a computational handshake or "Oracle," a precomputed resolution-path for non-aligned straddle cases, **ensuring** the instruction count for a $\mathcal{A}$-sector block remains **isochronous** regardless of its alignment relative to the Tier-1 registers. By **eliminating** the alignment-check penalty, the architecture **preserves** a deterministic execution window across the entire addressable space.

Here we describe the "Factory" math used to pre-calculate the Oracle tables, transforming a search problem into a static retrieval of pre-verified geometric facts.

3.3.1 Generating the Left and Right Hand Lookup Oracle

Part of the Oracle are tables of 1D vectors that map every possible bit-permutation of a Guard Shard to its boundary potential. These are pre-calculated *Finite State Automata* that eliminate runtime scanning [**?**].

- **Right-Hand Lookahead (RHL):** For any shard state S, the table stores the starting index of the longest contiguous zero-run terminating at the right-hand boundary. If the terminal bit is **1**, the value is **Null**.
- **Left-Hand Lookahead (LHL):** Maps the shard state to the final bit position of the zero-run starting from the left-hand boundary. If the initial bit is **1**, the value is **Null**.

3.3.2 Generating the Center Oracle (2D Matrix)

The Core Shard utilizes a 2D matrix ($2^{\mathcal{A}/4} \times \mathcal{A}/4$) to resolve internal gaps.

- **Mechanism:** Exhaustive enumeration of all $2^{\mathcal{A}/4}$ bit-patterns against all possible length requests within the core.
- **Math:** Each cell (i, j) is filled using a windowed zero-run algorithm. This transforms the shard into a precalculated *Finite State Automata* [**?**].

4 Deterministic Coalescence and the Occupancy Limit

Maintaining temporal determinism in a dynamic environment **requires** a deallocation process (coalescence) **mirroring** the efficiency of the allocation logic. Operational stability of the **Iscandar** lattice uses discrete enforcement of the Occupancy Limit(Φ), **precluding** the high-entropy states typically associated with conventional allocators [5].

4.1 Symmetric Coalescence Mechanics

Here, coalescence is not a variable-time search for adjacent free blocks, but a deterministic state update. Whether the Oracle descends to clear a bit-position or propagates the release upward through the hierarchy, the operation remains isochronous. Because the bit-address of any Sector (S_0) is a fixed function of its coordinates, the update to the occupancy counters (Γ) consumes the same instruction count (I_{base}) as the initial resolution. **The alignment is self-similar across all hierarchical depths, a property derived from the Structural Arity ($\mathcal{A}$) rather than a pre-defined set of size-classes.**

1. The atomic leaf node state Γ_0 is toggled to 0 (Available).
2. The change propagates immediately to the parent Row counter (Γ_1).
3. The Level-2 (Plane) and Level-3 (Cube) counters are updated through integer subtraction.

Because the lattice depth is fixed at $n = 3$, time complexity for allocation is identical to deallocation ($3I_{base}$). This symmetry ensures that system overhead remains constant regardless of memory "churn," eliminating the temporal spikes typically made by coalescing algorithms.

4.2 Operational Enforcement of Ω

The Occupancy Limit (Φ) **serves** as the authoritative boundary of the system's operational domain. While traditional allocators attempt to satisfy requests until the address space is 100% utilized—leading to $O(N)$ search patterns as fragmentation increases [7]—our lattice uses the Γ_3 counter as a deterministic "circuit breaker," depending on the memory request of the moment.

The decision logic for the Resolution Operator (Ω) is:

$$\Omega(\text{request}) = \begin{cases} \text{Resolve}(p, r, s) & \text{if } \Gamma_3 < K \\ \text{Refuse}(\text{Constant Time}) & \text{if } \Gamma_3 \geq K \end{cases} \tag{4}$$

where K is the sector threshold at which the Occupancy Limit is triggered ($K = \mathcal{A}^k \times (1 - \Phi)$)

4.3 Summary of Empirical Performance

This enforcement mechanism ensures the system never enters the "fragmentation trap." By refusing to allocate beyond the occupancy limit (Φ), **Iscandar** guarantees that the internal density of the $\mathcal{A}$-way branching structures never reaches a threshold where bit-mask steering degrades to a linear probe. This confirms a **flat temporal profile** until the moment of refusal.

4.4 Proof of Coalescence Invariant

Let Γ_{total} be the sum of all leaf nodes $S_{0,i}$. At any discrete time t, the hierarchical counters **must** satisfy the invariant:

$$\Gamma_{Apex} \text{ (a.k.a. } \Gamma_3) = \sum \Gamma_2 = \sum \Gamma_1 = \sum \Gamma_0 \tag{5}$$

Since deallocation is defined as:

$$\Gamma_x(t+1) = \Gamma_x(t) - 1 \quad \text{for all levels } x \in \{0, 1, 2, 3\}$$

along the specific resolution path, the invariant is **preserved**. Because this path is fixed at depth n, the lattice is restored to a consistent, deterministic state in nI_{base} time.

5 Observations

Earlier sections established **Iscandar** as deterministic within the primary scope of this paper. However, it exhibits properties warranting distinct architectural observations.

5.1 As an Extension to Binary Buddy Systems

The binary buddy system, pioneered by Knowlton [8] and Knuth [9], sought to minimize search latency through a power-of-two partitioning scheme ($R = 2$). While this approach facilitates efficient coalescence, its search complexity remains fundamentally constrained to $O(\log_2 n)$. As modern memory address spaces scale alongside processor word sizes, the increasing logarithmic depth of these trees drives significant latency. Furthermore, at high utilization, binary buddy systems trigger "stalling" behaviors; the search algorithm must traverse increasingly fragmented leaf nodes, which inevitably erodes deterministic performance.

We generalize buddy system principles to an **Arbitrary Radix** ($R = \mathcal{A}$). By shifting from a binary tree to a high-radix fractal lattice, we collapse the depth of the search space to a fixed architectural constant n. This structural shift moves memory management from the domain of probabilistic searching to the domain of **Geometric Coordinate Resolution**.

5.2 As a Symmetric Voronoi Partition

As detailed in the Architectural Axioms (Section 2.1), this lattice functions as a **Symmetric Voronoi Partition**, where the adjacency of "buddies" is a pre-baked geometric property of the address space rather than a calculated proximity, as shown in Section 5.2.

In this model, adjacency is defined by a partition's total volume **associating** with its handle (a wrapped pointer resolved in Section 9.4) rather than the Euclidean distance of individual points from a centroid. rather than the Euclidean distance of individual points from a centroid, as established in the geometric rules of the address space.

In a standard Voronoi partition, boundaries are dynamically computed as a function of the Euclidean distance to a specific "seed" or generator point. In the Iscandar lattice, while the "seed" is represented by the wrapped pointer, the partition boundaries are not computed at runtime, as defined by the invariant geometric properties of the lattice established in the Observations

section. Instead, they are architecturally invariant, defined strictly by the Structural Arity ($\mathcal{A}$) and the hierarchical depth (k) established in the Axioms. This replaces the need for distance-based calculations with a pre-baked, static coordinate resolution:

$$\text{Boundary}_{\text{Iscandar}} = f(\mathcal{A}, k) \neq f(\text{dist}(p, \text{seed})) \tag{6}$$

5.3 As Linear Programming

The operational stability of the system is maintained via an Interior Point Method—analogous to projective transformations used in Karmarkar's algorithm [10]. By enforcing a **O**ccupancy Limit (Φ), we **ensure** the resolution path avoids "edges" of the feasible region (the fragmentation trap). Here, the allocator does not merely manage space; it **navigates** a high-dimensional manifold where the floor serves as a protective barrier against non-deterministic search behaviors.

5.4 Amortized Performance: Constant Time

The **Iscandar** architecture utilizes a *Seamless Sack* model that achieves amortized constant time for memory operations through a Next-Fit Circular Cursor mechanism.

- **Mechanism:** The cursor maintains forward momentum across the $\mathcal{A}^k$-sector manifold. By never re-scanning fragmented entropy within a single "lap," the system distributes the metabolic cost of search across its entire operational lifecycle.
- **Principle:** The system offsets the latency of filtering fragmented sectors by performing high-efficiency, contiguous allocations in reclaimed or vacant regions.

5.5 Worst Case Performance: Sub-Linear Time

The architecture maintains determinism through hierarchical skip-logic at intermediate levels. This mechanism constrains the worst-case search complexity to the k-th root of the total lattice volume, where k represents the depth of the fractal hierarchy.

- **Mechanism: Dimensional Collapse.** The architecture maps the 1D address space to a k-dimensional manifold ($\mathcal{A}^k$). Metadata sentinels at the Plane and Row levels enable the search engine to bypass saturated volumes without bit-level interrogation.
- **Logic for the Geometry:**
  1. **Plane-Skip:** Evaluates $\mathcal{A}^2$-sector planes; skipping when free space is less than request.
  2. **Row-Skip:** Evaluates $\mathcal{A}$-sector rows; skipping when row density proves prohibitive.
  3. **Bit-Scan:** The engine restricts final verification to a single bit-vector of magnitude $\mathcal{A}$.
- **Ceiling:** For a lattice of volume $N = \mathcal{A}^k$, the architecture caps the maximum investigation path at $k \times \mathcal{A}$, limiting the worst-case cost to $k \times \mathcal{A}$ operations.

5.6 As Supporting Six Sigma Design

99 As detailed in the appendices, the compactness of the **Iscandar** kernel enables **complete path coverage testing** that operates in tandem with the previously established formal methods.

5.7 Scalability via Depth Extension

The **Iscandar** architecture scales by increasing the lattice depth up to two additional levels. Because the resolution time is a linear function of depth ($k \times I_{base}$), extending the lattice to $k = 4$ or $k = 5$ provides a massive increase in addressable memory volume while maintaining a strictly deterministic and predictable execution profile.

5.8 As Supporting Parallel Execution

The architecture allows for the distribution of the management load through parallel instantiation. Lesser structures can be managed by hardware subroutines to distribute the processing requirements across multiple units. These parallel hardware components can also supervise local memory independently, communicating management data upward through the hierarchy only as required.

6 Conclusion

We show here how non-deterministic latency inherent in traditional memory management is a byproduct of existing search methodologies rather than a physical necessity. Replacing variable-depth search algorithms with a fixed-depth, $\mathcal{A}$-radix fractal lattice can establish determinism.

7 Glossary of Terms

Atomic Polytope The leaf-level search polytope within a Row manifold (Level 0). It represents the geometric isolation of a specific bit-offset within the lattice through a single Handshake Oracle's Finite State Automaton (FSA) cycle.

Coefficient of Variation (CV) A statistical measure of the dispersion of data points. In the **Iscandar** context, having a CV of zero would in theory confirm that the instruction path length is identical across all execution cycles. [7]

Contiguity Guarantee The formal assurance (Section 2.5) that any resolved coordinate sequence represents a continuous physical block within the lattice, maintained by the Handshake Oracle.

Global Registrar The Level 3 (Cube Scale) apex of the hierarchy. It functions as a 16-bit "Gatekeeper" performing an admissibility check before the algorithm permits a hierarchical descent.

Handshake Oracle (LHL/RHL/Center) A Finite State Automaton, being a precomputed body of solutions used at the Tiny (Level 0) and Small (Level 1) allocator levels to bridge "fractured" sequences between sectors. It utilizes Left-Hand Lookahead (LHL), Right-Hand Lookahead (RHL) and Center Lookahead tables to weld disparate fragments into a contiguous magnitude. In microcontroller implementations these are laid out as ROM tables.

Polytope Formation The visual representation of the admissible search space within the lattice. Each hierarchical tier (Cube, Plane, Row) can manifest polytopes that collapse as memory occupancy increases.

Occupancy Limit (Φ) The mandatory occupancy boundary (Section 4.2) that prevents the system from entering a high-entropy state. Once the floor is reached, the system issues a deterministic refusal rather than engaging in variable-time search.

8 Updating Memory State Spaces by Mating the Oracle Tables

This section describes how **Iscandar** resolves memory requests by intersecting partitioned subspaces. The system utilizes primary ratios relative to the structural Arity ($\mathcal{A}$): the Guard Shards ($0.5\mathcal{A}$) and the Core Shard ($0.25\mathcal{A}$).

Polytope Formation at Cube Level

The Cube Scale Allocator (Global Registrar)

Domain: Total Cube Volume resolution (addressable at $\mathcal{A}^k$).

Summation Predicate:

$$|C| = \sum_{i=0}^{\mathcal{A}-1} (\text{Sector Count of Plane } i)$$

Discussion: The Cube Scale Allocator computes the unique **"Gatekeeper"** sector count of the architecture. It provides the primary fast-fail heuristic for the system; if a request (S, M) is determined to be non-admissible at this tier, the algorithm immediately terminates without initiating a descent. This makes the registrar the most efficient filter in the hierarchy, ensuring that the cost of coordinate resolution is only paid when success is mathematically guaranteed.

Polytope Formation At Row and Plane Levels

The Large Resolution ($\mathcal{A} < L \leq 2\mathcal{A}$)

Domain: Row-join and sequence resolution within an $\mathcal{A}^2$-sector Plane manifold.

Summation Predicate:

$$C_{plane} = \sum_{i=0}^{\mathcal{A}-1} (C_{row,i} \cdot [y_i = 1])$$

Discussion:

- **Mechanism:** The *Structural Bridge*.
- **Geometric Event:** *Boundary Handshake* between adjacent vectors.
- **Resolution:** The system constructs the span via $P+T$. The Right-Hand Lookup (RHL) of Vector A (Preamble) mates with the Left-Hand Lookup (LHL) of Vector B (Tail) to verify connectivity across the boundary [**?**].

This tier represents a fractal expansion of the Level 1 logic. It treats individual rows as the atomic units of resolution, welding row-sequences to satisfy multi-row allocation requests. The Large allocator functions as a resolution engine, calculating a dynamic subset of Rows within the Plane to satisfy continuity requirements.

The Huge Resolution ($L > 2\mathcal{A}$)

Domain: Plane-join and sequence resolution within an $\mathcal{A}^k$ Cube manifold.

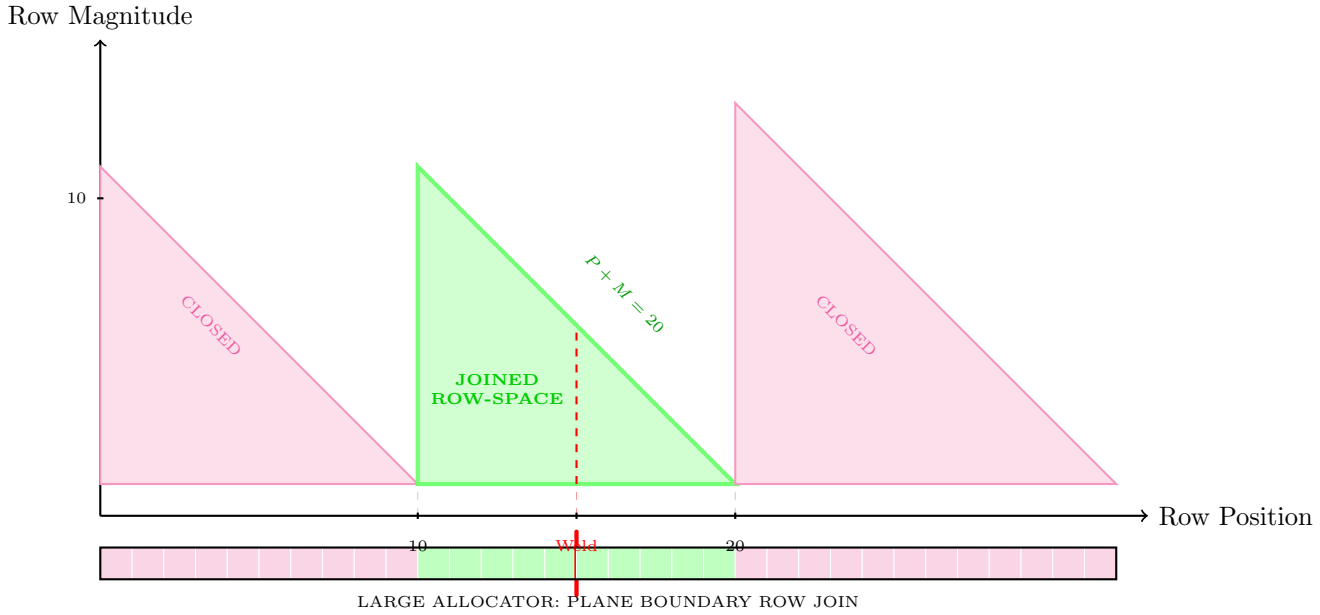
Summation Predicate:

$$C_{huge} = \sum_{i=0}^{\mathcal{A}-1} (C_{plane,i} \cdot [z_i = 1])$$

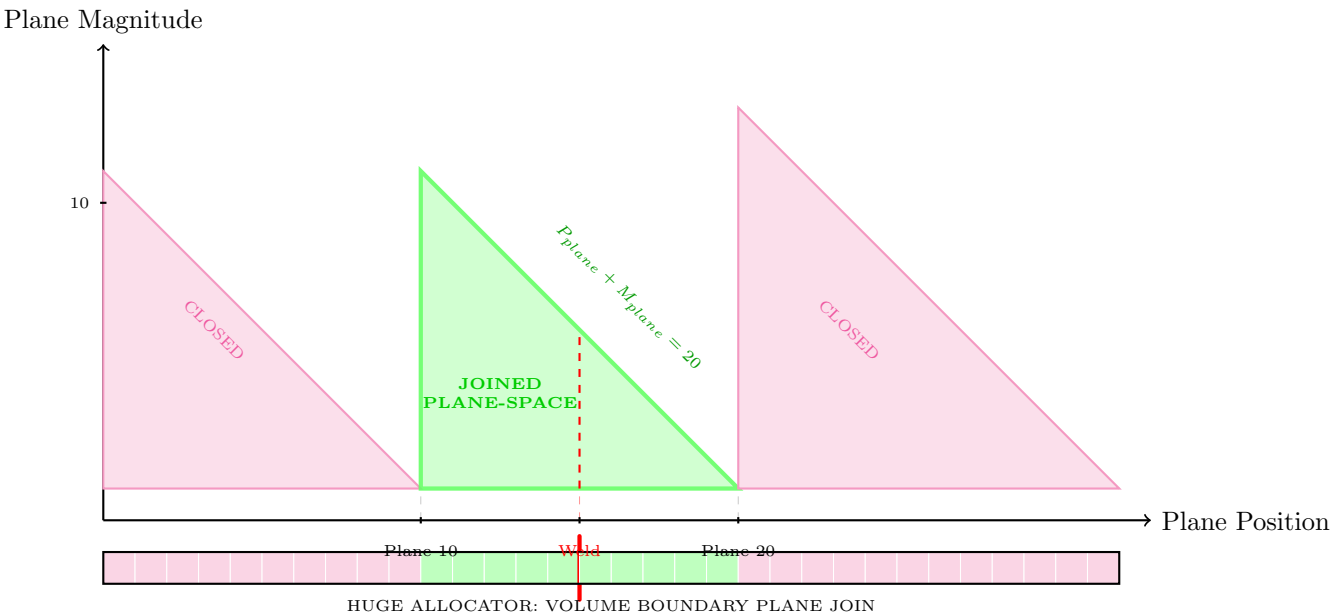
Discussion:

- **Mechanism: The Recursive Span**
- **Geometric Event:** *Multi-Vector Extrusion*
- **Resolution:** The system constructs the span via $P + (m \times \mathcal{A}) + T$. The Oracle verifies the m intervening words as "Clear Rows" ($\mathcal{A}$ nullity checks), treating the span as a single geometric object via induction [**?**].

This represents the final join-level resolution of the hierarchy. It manages the 32-radix plane counts, welding 2D plane segments into a unified 3D volume request. By treating planes as the fundamental unit of resolution at this tier, the system ensures that even the largest possible allocations are resolved with identical determinism. The Huge allocator calculates the specific number of Planes required to weld a unified 3D volume, treating each Plane count as a subordinate metric for the total Cube count.



LARGE ALLOCATOR: PLANE BOUNDARY ROW JOIN



HUGE ALLOCATOR: VOLUME BOUNDARY PLANE JOIN

Polytope Formation Between Rows

The Small Resolution ($L \leq \mathcal{A}$)

Domain: Sector-join and sequence resolution within an $\mathcal{A}$-sector Row manifold.

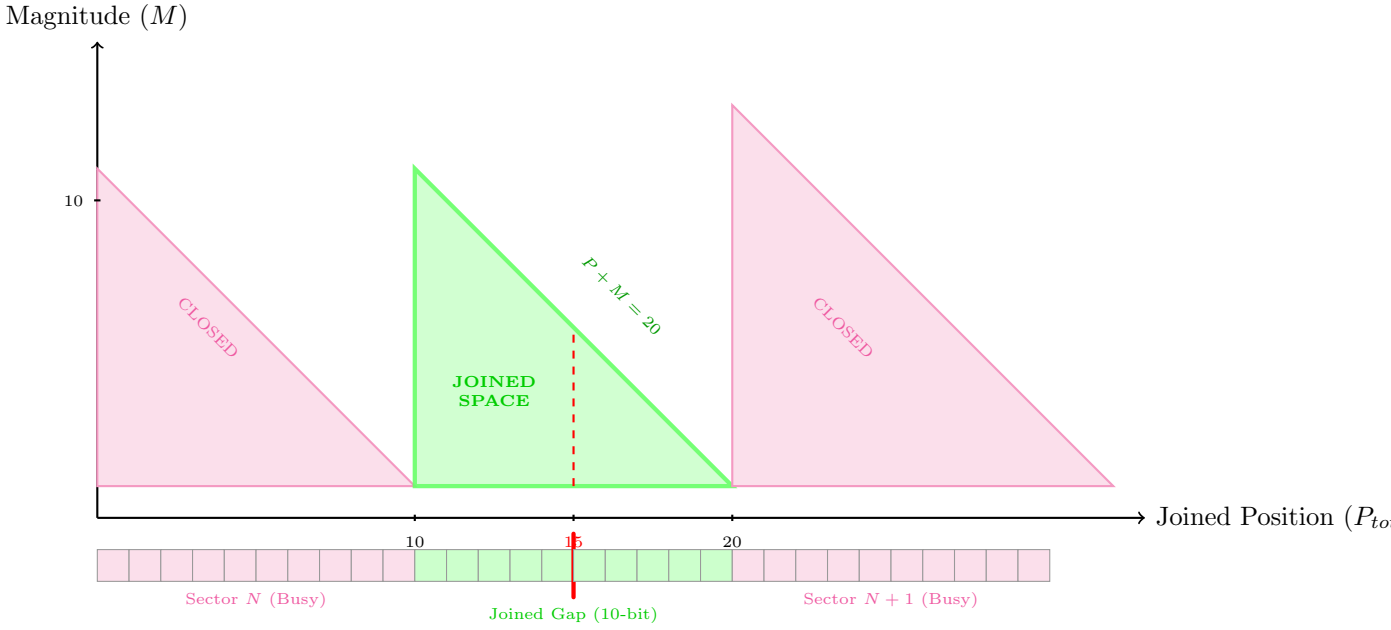
Summation Predicate:

$$C_{row} = \sum_{j=0}^{\mathcal{A}-1} (C_{bits,j} \cdot [x_j = 1])$$

Discussion:

- **Mechanism:** *Tripartite Join*
- **Geometric Event:** *Polytope Intersection*
- **Resolution:** The Oracle pieces together the Right-Hand Lookahead (RHL) of the first Guard, the internal gaps of the Center Lookahead, and the Left-Hand Lookahead (LHL) of the second Guard, to verify a contiguous span of available sectors. [?]

This tier functions as the primary submachine for the bottom level of welding, utilizing the Oracle to join partial sector tails and heads into a single contiguous magnitude. By treating the row as a unified geometry, boundary seams become transparent to the resolution logic.



Polytope Formation Within Row

The Tiny Resolution ($L \leq 0.25\mathcal{A}$)

Domain: The bit-lattice contained within a single Row manifold.

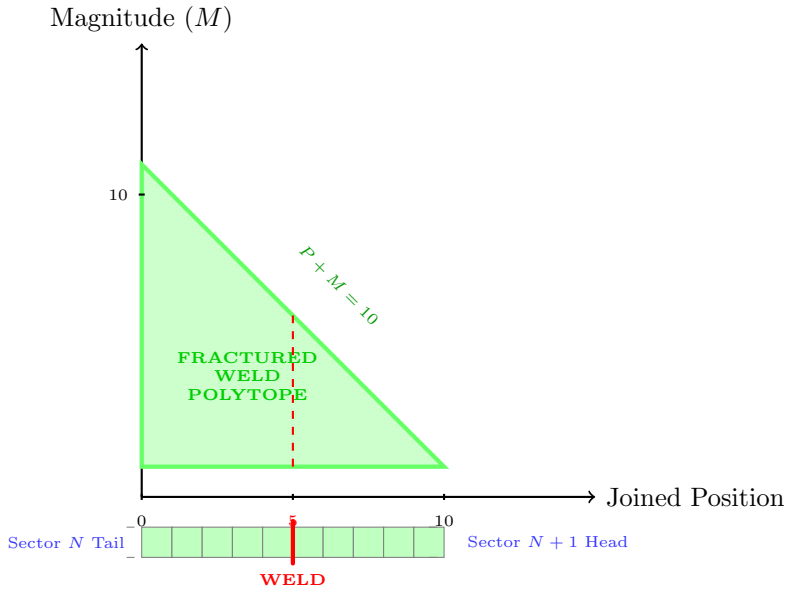
Summation Predicate:

$$C_{bits} = \sum_{i=0}^{\mathcal{A}-1} (\neg b_i \cdot [w_i = 1])$$

Discussion:

- **Mechanism:** The request falls entirely within the Core Shard (S_c).
- **Geometric Event:** An *Interior Locality* query.
- **Resolution:** The 2D Oracle Table functions as a point-in-polytope calculator. The system requires only the center lookup to return the $(position, length)$ tuple [**?**].

This represents the leaf-level execution of the deterministic descent. The **Oracle Center look-up table** identifies contiguous zero-bit sequences within a sector. This forms the Atomic Polytope of a search, resolving the physical bit-offset in a fixed hardware cycle.



9 Description of Instruction Groups

We map each instruction group to a specific predicate of the **Iscandar** hierarchy established in the Architectural Axioms. This ensures the hardware execution is a transparent reflection of the underlying mathematics. Here, we assume a 32-radix descent where each level acts as a "Gate" that must be cleared to reach the subsequent tier.

9.1 Phase I: The Admissibility Gate (Level 3)

Predicate: $L \in \Gamma_{Apex} \iff \Gamma_{Apex} \leftarrow \Gamma_{Apex} + L$

1. **Instruction Group 1:** Load Global Registrar: Fetch the 16-bit register located at the Apex sector address as defined in the Cube Scale Allocator.
2. **Instruction Group 2:** Admissibility Comparison: Compare against the requested magnitude L. If False: Immediate Exit (Fast-Fail) as required by the Global Registrar's heuristics. If True: Increment by the requested magnitude.

9.2 Phase II: The Plane-Level Resolution (Level 2)

Predicate: $P_{Plane} \iff \sum_{i \in Plane}(Row[Inclusion]) \geq L$

3. **Instruction Group 3:** Index Plane Metadata: Using the address bits, index into the 32-radix plane metadata table.
4. **Instruction Group 4:** Load Row-Summary: Fetch the 15-bit metadata for the target plane summarizing the 32 constituent rows as established in Axiom II.
5. **Instruction Group 5:** Candidate Selection: Evaluate the plane-state to identify the first row-sequence that satisfies the weld.
6. **Instruction Group 6:** Heuristic Branch/Scan: Evaluate the 32-radix plane metadata scan to confirm the steering vector.

9.3 Phase III: The Row-Level Weld (Level 1)

Predicate: $Weld \iff RHL(Row_n) + Intermediates + LHL(Row_{n+m}) \geq L$

7. **Instruction Group 7:** Leading Fragment Access (RHL): Perform a Look-Up Table (LUT) lookup on the starting row's tail.
8. **Instruction Group 8:** Trailing Fragment Access (LHL): Perform a LUT lookup on the final row's head.
9. **Instruction Group 9:** Scalar Summation: Add Lead + Intermediates + Trail, maintaining the Contiguity Guarantee.
10. **Instruction Group 10:** Weld Validation: Compare the result against M to finalize the inter-row continuity.

9.4 Phase IV: The Bit-Lattice Isolation (Level 0)

Predicate: $P_{Tiny} \iff \sum_{i=start}^{end} \neg b_i = L$

11. **Instruction Group 11:** Center LUT Index: Access the bit-occupancy table for the target sector.
12. **Instruction Group 12:** Triangle Projection: Map the LUT result to the physical bit-offset, resolving the Atomic Polytope within the Row manifold.
13. **Instruction Group 13:** Commitment/Return: Write the new occupancy to the 15-bit metadata and return the start address S.

10 Desk Check Summary: The Tiny Search ($L \leq 8$)

Here we posit a hypothetical 32-bit machine providing the data structures needed to demonstrate one example through the Instruction Groups, on possible "Tiny Search" tracing the sequence needed to resolve an allocation request L of 8 sectors or fewer.

1. **Admissibility Check:** The Global Registrar compares the request size L against the aggregate occupancy (Γ) and entropy thresholds.
2. **Plane Navigation:** The Rotational Steering Vector ($\vec{R}$) is applied at the Cube level to select a candidate Plane.
3. **Metadata Retrieval:** The 32-bit Row-Summary mask for the selected plane is loaded into the register.
4. **Row Isolation:** A mask check identifies the first available Row index within the plane summary.
5. **Sector Load:** The 32-bit data word representing the candidate Row is fetched.
6. **Hierarchical Bisection:** The 32-bit row is divided into two 16-bit "Half-Words" to initiate the Oracle probe.
7. **Oracle (High):** A LUT probe returns the Right Hand Lookahead for high 16 bits.
8. **Oracle (Low):** A LUT probe returns the Left Hand Lookahead for low 16 bits.
9. **Boundary Intersection:** The system calculates the "Center-Straddle" by summing the RHL of the high half and the LHL of the low half.
10. **Core Shard Interrogation:** If the straddle fails, the 32-bit row is treated as four 8-bit quadrants for unrolled sequential validation.
11. **Geometric Resolution:** The successful bit-run is mapped to a physical offset using the Triangle Projection method.
12. **Lattice Commitment:** The new occupancy state is masked into the row, and local metadata is updated.
13. **Deterministic Exit:** The system returns the resolved pointer.

11 Volumetric Analysis

The following analysis provides the empirical verification of the **Iscandar** architectural thesis. Using a **Radix-32** sentinel-steered manifold, the system shows determinism across the entire addressable sectors (32768). The data confirms that computational overhead remains decoupled from the scale of the memory system, ensuring that pointer resolution is invariant to heap saturation.

11.1 Methodology

A stress-test audit was performed utilizing a $K = 3$ fractal depth configuration. The system was subjected to sequential allocations, with performance metrics sampled at every 50th operation to observe the behavior of the manifold under heavy saturation.

- **Memory Request (Sectors).**
- **Memory Touches (Sectors):** The total register-level interactions required to update the bitmask and sentinel hierarchies (`RMASK`, `PMASK`).

11.2 Test Data (generated from Section 10)

Table 1: **Iscandar** Performance Metrics ($A = 32, K = 3$)

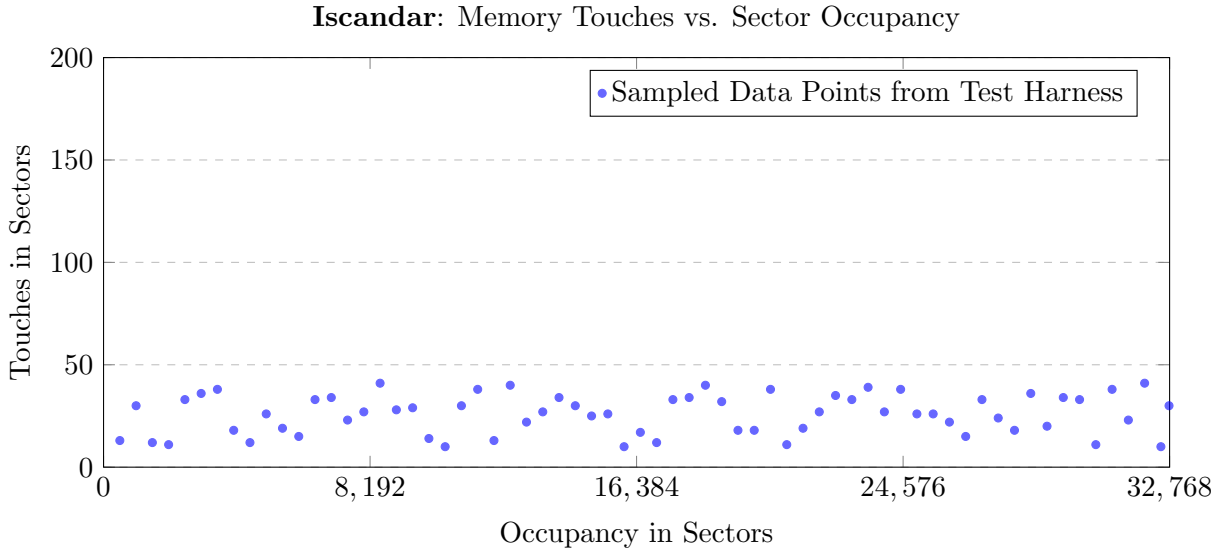
Occupancy (Sectors)	Memory Request (Sectors)	Memory Touches (Sectors)
50	4	33
100	3	36
150	5	32
200	6	37
250	5	13
300	6	11
350	4	12
400	5	37
450	4	11
500	4	13
550	6	32
600	6	11
650	4	26
700	3	31
750	2	20
800	6	25
850	3	41
900	5	33
950	2	36
1000	6	30
1050	6	37
1100	6	33
1150	6	10
1200	4	15
1250	5	36
1300	2	19
1350	2	17
1400	2	23
1450	6	41
1500	5	12
1550	5	32

Table 1 – Continued from previous page

Occupancy (Sectors)	Memory Request (Sectors)	Memory Touches (Sectors)
1600	5	17
1650	5	12
1700	6	36
1750	2	27
1800	2	12
1850	2	20
1900	4	13
1950	3	12
2000	5	11
...	...	...
32650	5	33
32700	3	11
32750	4	30

11.3 Graphical Verification

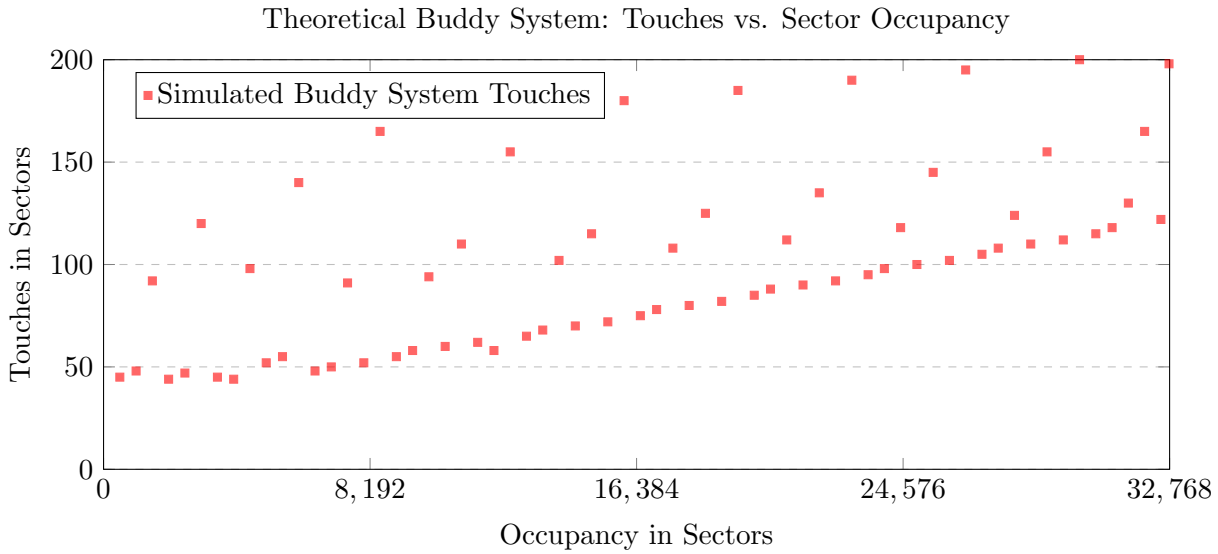
Figure 11.3 documents the stability of **Iscandar** across its full volumetric range. By sampling data at 500-sector intervals, the visualization maintains the characteristic "flat" trendline of the Radix-32 steering logic, confirming determinism as the system approaches the 32,768 sector limit.



Distribution demonstrating bounded search for sectors.

11.4 Buddy System Comparison Simulation

To contrast, a theoretical binary buddy system was simulated for a heap size comparable to ours, a 128 KB heap utilizing 4-byte sectors. This needs a binary tree depth of $D = 15$. Figure 11.4 illustrates the search effort inherent in recursive block splitting and coalescing.



Simulated binary buddy allocation at 4-bytes per sector in a 32768 sector memory pool.

12 Iscandar: Laboratory Source Code

This DYALOG APL implementation was used to generate the report's tables. The test code simulates high-churn volumetric stress to verify determinism of the sentinel-steered geometry.

```
    ⍝⍝⍝
```

```
       r_idx   ((p-1) A )+r
       SCAN_OVERHEAD + 1    Row Gate Tax (Macro G05)
       :If (r_idx + rows_needed - 1)  1024
         check_range   RMASK[r_idx + rows_needed - 1]
         SCAN_OVERHEAD +  check_range
         :If  / check_range = 0
           row_start   ((r_idx-1) A ) + 1
           LATTICE[row_start + Size - 1]   1    Atomic Write (G08)
           SCAN_OVERHEAD + 1
           RMASK     / (1024 32   LATTICE)
           PMASK     / (32 32    RMASK)
           Z   row_start Size   found    1    :Leave
         :EndIf
       :EndIf
     :EndFor
   :EndIf
   :If found   :Leave    :EndIf
 :EndFor


   --- MODULE 4: ENTROPY REVERSAL (DEALLOCATION) ---
   FREE_K13 Data;idx;sz
 :If 0 =   ,Data    :Return    :EndIf
 (idx sz)    ,Data
 LATTICE[idx + sz - 1]    0
 RMASK      / (1024 32   LATTICE)
 PMASK      / (32 32    RMASK)


   --- MODULE 5: STATISTICAL AUDIT HARNESS ---
   RUN_LAB;SACK;sz;res;history;occ;c;stats;t;m;s;idx;v
 INIT_GEOMETRY   SACK           history    0 3 0     occ    0
 :For c :In  500
     sz    1 + (?1024) + (0 = ?10)  ?14000
     res    ALLOC_K13 sz
     :If res   0
         history   history    occ sz SCAN_OVERHEAD
         SACK ,   res      occ +   sz
     :EndIf
     :If (0 = 5 | c)   ( SACK ) > 0
         idx   ? SACK      FREE_K13  SACK [idx]
         occ -  ( SACK [idx])[2]    SACK    ((  SACK  )    idx) / SACK
     :EndIf
 :EndFor
        ' Usage (Sect) | Request | Touches '
        (-50    1  history  )    history
 t    history[;3]   m    (+/t) ( t ) 1
 s    (((+/(t-m)*2) ( t ) 1 )*0.5)    v    s*2
        '--- AUDIT COMPLETE ---'
        'MEAN OVERHEAD (METABOLIC COST): ',m
        'SIGMA (JITTER): ', s            'VARIANCE: ', v

```

Listing 1: ISCANDAR 16.4.1 Lab Implementation

References

[1] D. E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms.* Addison-Wesley, 3rd Edition, 1997.

[2] K. C. Knowlton. "A Fast Storage Allocator." *Communications of the ACM*, Vol. 8, No. 10, pp. 623-624, 1965.

[3] R. G. Dromey. *A Bit-Block Representation for Constant-Time Memory Management. Software: Practice and Experience*, 1980s.

[4] *Ibid.*

[5] N. Karmarkar. "A New Polynomial-Time Algorithm for Linear Programming." *Combinatorica*, 4 (4): 373-395, 1984.

[6] *Ibid.*

[7] *Ibid.*

[8] K. C. Knowlton, *op. cit.*

[9] D. E. Knuth, *op. cit.*

[10] N. Karmarkar, *op. cit.*

[11] Edelsbrunner, H., & Harer, J. (2010). *Computational Topology: An Introduction.* American Mathematical Society.

[12] Ziegler, G. M. (1995). *Lectures on Polytopes.* Springer-Verlag.

[13] Crochemore, M., & Rytter, W. (2002). *Jewels of Stringology.* World Scientific Publishing.

[14] Alexandrov, P. S. (1959). *Combinatorial Topology.* Dover Publications.